

DESIGN AND IMPLEMENTATION OF A POLICY-CONTROLLED ON-PREMISE AI WORKBENCH FOR CONFIDENTIAL WORKFLOWS

A Synopsis

Submitted

In Partial Fulfillment of the Requirements

For the Degree of

Bachelor of Technology (B. Tech.)

in

Computer Science & Engineering

by

Bhumik Sharma

(2401920100127)

Harsh Kumar Singh

(2401920100152)

Harsh Kumar

(2401920100151)

Harshit Kumar

(2401920100155)

Under the Supervision of

Ms. Abha Kaushik

Assistant Professor



**G.L. BAJAJ INSTITUTE OF TECHNOLOGY & MANAGEMENT
GREATER NOIDA**



**DR. A P J ABDUL KALAM TECHNICAL UNIVERSITY,
UTTAR PRADESH, LUCKNOW**

2026-2027

1. Introduction

The recent advancement in Large Language Models (LLMs), Vision-Language Models (VLMs), and autonomous multi-step agents has fundamentally transformed organizational information processing, document synthesis, and computational reasoning. Commercial AI services hosted on public cloud infrastructures demonstrate remarkable capability in conversational understanding, multimodal synthesis, and automated code generation. However, deploying public cloud AI APIs within security-sensitive environments—such as industrial manufacturing facilities, public sector offices, defense units, research laboratories, and healthcare organizations—introduces critical vulnerabilities. Transmitting proprietary documents, operational schemas, and internal records to external third-party servers presents severe risks of data exfiltration, regulatory non-compliance under the Digital Personal Data Protection (DPDP) Act 2023, ISO 27001 violations, and loss of intellectual property sovereignty.

Concurrently, the rapid evolution of open-weight foundation models (such as Meta Llama 3, Qwen 2.5, Mistral, and Phi-3), paired with efficient local inference engines (Ollama, llama.cpp, and vLLM), has created a compelling opportunity to execute state-of-the-art AI workloads entirely on-premise without relying on external cloud APIs. Despite these advances, simply deploying an open-source model behind a generic chat interface fails to solve real enterprise challenges. Modern operational workflows require ingesting scanned, heterogeneous reports; extracting tables and layout data with high fidelity; retrieving factual internal records without hallucinations; executing computational analysis safely; and enforcing strict procedural governance over autonomous agent tool interactions.

This project, titled '*Design and Implementation of a Policy-Controlled On-Premise AI Workbench for Confidential Workflows*', presents an integrated, secure, self-hosted platform engineered specifically for air-gapped organizational environments. Rather than attempting to train a foundation model from scratch, the primary engineering contribution of this project is the design and realization of a multi-tiered control architecture around local open-weight models. The system synthesizes capability-based routing, multimodal document parsing, local dense Retrieval-Augmented Generation (RAG), bounded finite-state agent execution, isolated code sandboxing, and deterministic verification to deliver an auditable, dependable, and strictly contained intelligent workspace.

2. Objectives

- **Air-Gapped On-Premise Execution:** To architect and deploy a completely self-hosted AI workbench operating on local infrastructure with verified zero runtime external network dependencies or telemetry egress.
- **Deterministic Capability and Policy Routing:** To develop an intelligent classifier and policy router that deterministically categorizes incoming requests (e.g., text reasoning, document extraction, internal search, sandboxed data computation) and routes them to specialized local processing pipelines.
- **High-Fidelity Multimodal Ingestion:** To implement a robust document parsing pipeline combining digital text extraction, local Optical Character Recognition (OCR using PaddleOCR/Tesseract), and Vision-Language Models (VLMs) to process complex scans, blueprints, and tabular spreadsheets while preserving document hierarchy and page provenance.
- **Evidence-Grounded Local Knowledge Retrieval (RAG):** To build a localized vector search subsystem utilizing semantic chunking, dense embeddings, vector indexing, and cross-encoder reranking to ensure answers are strictly linked to verified internal organizational documents.
- **Bounded Finite-State Agent Orchestration:** To implement an autonomous agent engine governed by explicit finite state machines, tool allowlists, strict step limits, and timeout boundaries to eliminate runaway loops, excessive agency, and unpredictable model behaviors.
- **Isolated Code and Data Analysis Sandbox:** To engineer an isolated execution environment with disabled network interfaces, restricted CPU/RAM quotas, and restricted filesystem namespaces for safely executing LLM-generated Python and data analysis routines.
- **Deterministic Verification and Human Oversight:** To integrate automated schema validation, calculation checks, and confidence scoring, coupled with a mandatory human-in-the-loop review workflow for high-risk or low-confidence operational outputs.
- **Runtime Observability and Verifiable Auditability:** To establish end-to-end structured audit logging, pipeline traceability, and network isolation validation to provide cryptographic accountability for every generated deliverable.

3. Existing System (Application Based Project / Related Work)

In modern organizational workflows, existing computational solutions can be broadly categorized into commercial cloud AI platforms, open-source local chat interfaces, and experimental agent frameworks. An analysis of 5 prevailing industry solutions highlights foundational technical shortcomings that our project resolves:

Existing System	Key Features & Architecture	Critical Limitations in Sensitive Workflows
Commercial Cloud AI (ChatGPT Enterprise, Claude, MS Copilot)	Centralized proprietary LLMs hosted on public cloud. Massive parameter scale, high zero-shot capability, API-based integration.	Mandatory external data transmission; risk of data residency breaches; recurring token costs; non-compliance with air-gapped security protocols.
Local Model Frontends (Open WebUI, Ollama Web, LM Studio)	Desktop or web wrappers running quantized GGUF/AWQ models locally via Ollama or llama.cpp for single-turn prompt chat.	Lack multi-agent orchestration; no deterministic policy routing; rudimentary RAG lacking reranking; unable to execute sandboxed code safely.
Local Document RAG (PrivateGPT, LocalGPT, AnythingLLM)	Document ingestion via LangChain/LlamaIndex, chunking, and local embedding retrieval into ChromaDB/FAISS.	Vulnerable to OCR degradation on scanned images; lacks multimodal VLM extraction; no calculation verification; absent human approval gates.
Autonomous Agent Frameworks (AutoGPT, CrewAI, LangGraph)	Multi-agent prompting patterns allowing LLMs to invoke external bash tools, web search, and script interpreters.	Risk of unconstrained agency and prompt injection; tools run in host environment without container isolation; prone to non-terminating loops.
Enterprise Data Suites (Dataiku, IBM Watsonx On-Prem)	Comprehensive enterprise data management, model registry, and workflow orchestration suites.	Extremely high licensing costs; heavy operational overhead requiring massive distributed clusters; not optimized for agile single-node workstations.

Synthesis of Related Academic Research:

The architecture builds upon foundational paradigms across multiple computing domains. In Retrieval-Augmented Generation, Lewis et al. (2020) and Gao et al. (2023) demonstrated that dense vector indexing combined with semantic chunking significantly reduces generative hallucinations by conditioning model inference on authoritative retrieved passages. In agentic reasoning, Yao et al. (2023) established the ReAct (Reasoning + Acting) paradigm, while Schick et al. (2023) introduced Toolformer to demonstrate self-taught tool utilization; however, both approaches exhibit safety vulnerabilities when operating with unconstrained execution environments. Recent investigations into LLM safety by Greshake et al. (2023) proved that untrusted document text can execute indirect prompt injection attacks on autonomous agents, making hardened tool allowlists and isolated sandboxes non-negotiable. Furthermore, advancements in lightweight 4-bit and 8-bit quantization by Dettmers et al. (2023) enable high-throughput parameter-efficient inference on standard edge hardware, validating the technical feasibility of our fully on-premise workbench.

4. Motivation & Problem Statement

4.1 Motivation

Modern organizations handle immense volumes of highly proprietary, mission-critical assets, including technical blueprints, industrial equipment inspection reports, confidential legal contracts, audited financial records, and medical diagnostics. While staff could achieve immense productivity gains through generative summarization, automated report drafting, and programmatic data querying, enterprise security policies and national legal frameworks strictly forbid transmitting confidential files to public multi-tenant AI servers. Commercial AI subscriptions further impose unpredictable per-token pricing, vendor lock-in, and sudden service deprecation.

Furthermore, organizations in manufacturing, civil infrastructure, and administrative governance are frequently subject to stringent air-gapped security protocols where computing nodes cannot maintain constant internet connectivity. Consequently, there is an urgent imperative for an on-premise, policy-controlled computational workbench that empowers knowledge workers with advanced AI capabilities while guaranteeing complete data sovereignty, predictable operational expenditure, and zero operational leakage.

4.2 Problem Statement

The fundamental engineering challenge addressed in this project is: *How can an organization leverage cutting-edge multimodal AI, knowledge retrieval, and agentic task execution entirely within a local, air-gapped environment while maintaining deterministic control over confidential information, preventing hallucinated outputs, bounding autonomous tool actions, and containing untrusted code execution?*

To achieve a dependable system, the engineering implementation must systematically eliminate four critical failure modes identified in conventional AI systems:

- **Data Sovereignty & Containment Failure:** The risk of sensitive organizational documents leaking through outbound API calls, model telemetry, or third-party cloud caching. The system must enforce zero external network egress at runtime.
- **Generative Hallucination & Ungrounded Reasoning:** The tendency of foundation models to generate factually erroneous numbers or fabricate non-existent engineering standards. The system must enforce strict retrieval grounding and citation linking.
- **Unbounded Agent Autonomy:** The danger of recursive prompt chaining leading to runaway computational loops, destructive system operations, or non-terminating agent steps. The system must govern agent execution with strict finite-state invariants and step quotas.
- **Host Compromise via Untrusted Code Execution:** The security hazard of executing LLM-generated Python or shell scripts directly on the host operating system. The system must encapsulate all execution within an isolated, resource-constrained sandbox.

5. Proposed Methodology & Process for Implementation

5.1 Detailed Explanation of the Proposed Solution

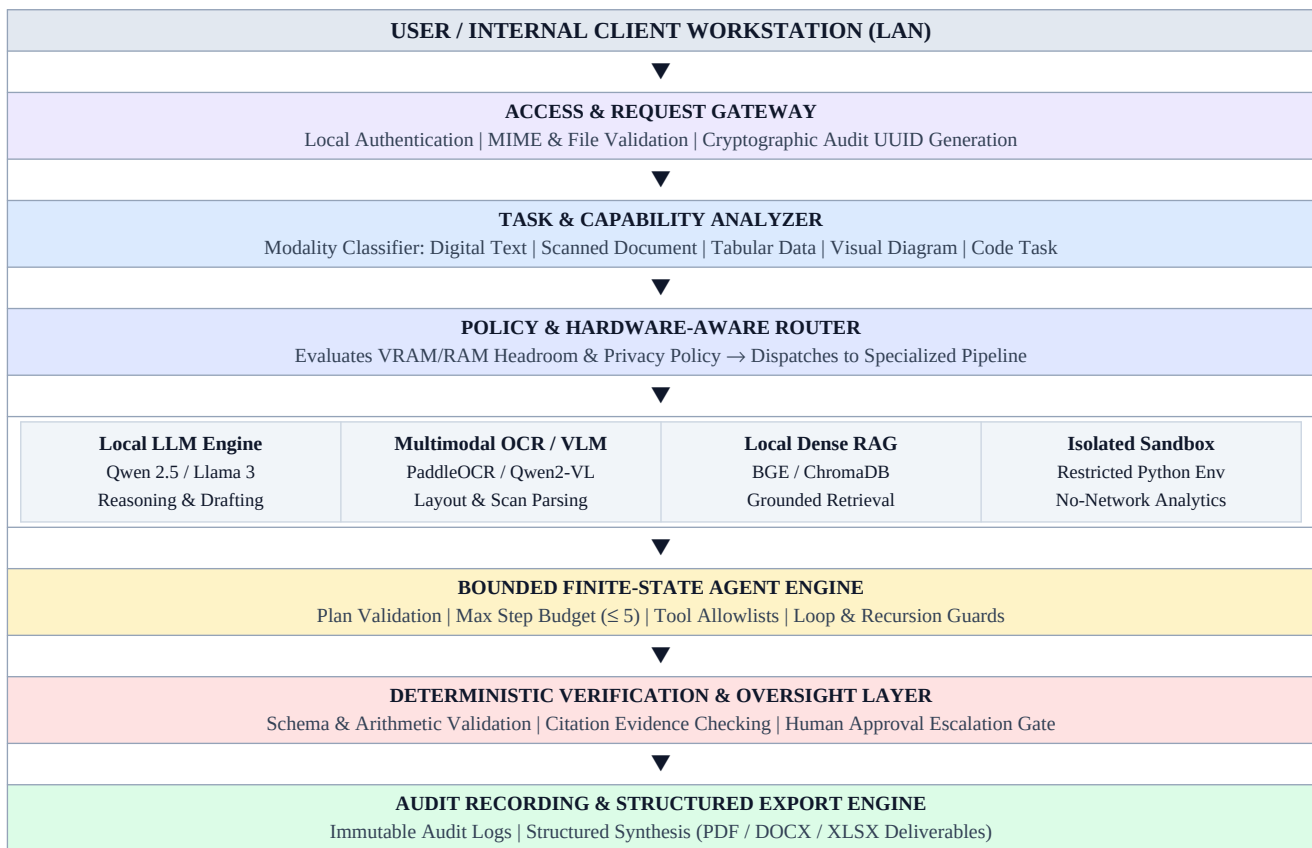
The proposed solution is an integrated, policy-governed on-premise AI workbench deployed inside an organization's internal local area network (LAN). Rather than exposing a raw, unconstrained language model directly to user prompts, the system establishes a defense-in-depth architecture. The platform treats every incoming request as an untrusted transaction that must be deterministically analyzed, authorized, routed to specialized micro-services, executed under bounded invariants, and mathematically verified before any deliverable is surfaced to human operators.

The system architecture comprises eight tightly integrated functional layers:

- **Layer 1 — Access & Request Gateway:** Enforces role-based access control, validates MIME types, scans file signatures, and binds each session to an immutable cryptographically secure UUID for complete operational auditability.
- **Layer 2 — Task & Capability Analyzer:** Employs deterministic rule classifiers and shallow text embeddings to detect required modalities: Digital Text, Scanned PDF, Tabular Data, Engineering Diagram, or Code Transformation.
- **Layer 3 — Policy & Hardware-Aware Router:** Dynamically inspects active GPU VRAM and CPU load. It routes simple text tasks to lightweight quantized models, visual documents to local VLMS, and data calculations to the sandbox.
- **Layer 4 — Bounded Agent Execution Engine:** Formulates task execution as a deterministic finite state machine (FSM). The LLM plans steps, but execution is governed by hard step caps (≤ 5 steps), tool allowlists, and execution timeouts.
- **Layer 5 — Local Knowledge Pipeline (Dense RAG):** Ingests enterprise manuals, creates dense vector embeddings via local transformer models, indexes them into a local vector store, and applies cross-encoder reranking to ensure factual grounding.
- **Layer 6 — Isolated Execution Sandbox:** A hardened subprocess container with network loopback disabled, non-root user execution, and strict memory/CPU quotas for executing data-transformation scripts.
- **Layer 7 — Verification & Human-in-the-Loop Oversight:** Executes automated JSON schema validation, arithmetic checks, and citation coverage tests. Low-confidence outputs trigger a mandatory human approval gate.
- **Layer 8 — Observability & Deliverable Export:** Generates structured forensic audit records and exports publication-ready documents in PDF, DOCX, and XLSX formats.

Core Architectural Philosophy: The platform operates on the principle of *least privilege for autonomous models*. By enforcing strict physical separation between model reasoning, file retrieval, code execution, and deliverable rendering, the architecture mathematically eliminates the possibility of uncontained agent behaviors, indirect prompt injection breaches, and remote telemetry leakage.

5.2 System Architecture and Modular Flow



End-to-End Operational Execution Lifecycle:

The architecture follows a strict synchronous execution flow to maintain absolute determinism and security:

- Ingress & Sanitization:** The request is sanitized, checked against file size and format allowlists, and assigned a session audit identifier.
- Capability Routing:** The analyzer inspects file extensions, headers, and semantic prompts to select the target model runtime.
- Multimodal Extraction & Grounding:** Scanned images pass through local OCR while textual queries trigger semantic vector search over internal manuals.
- Bounded Agent Reasoning:** The agent plans sub-actions within strict state-machine constraints. Tools cannot invoke unauthorized shell commands.
- Hermetic Sandbox Execution:** Any analytical code is piped to an isolated subprocess with networking disabled and tight memory ceilings.
- Verification & Export:** Output claims are checked against retrieved text snippets, formatted into structured deliverables, and logged for audit compliance.

5.3 How the Solution Addresses the Identified Challenges

Problem / Requirement	Proposed Technical Mechanism	Expected Engineering Outcome
Sensitive corporate data must never leak to third parties	Full on-premise execution using local open-weight models (GGUF/vLLM) and air-gapped local storage with zero runtime cloud APIs.	Complete mathematical elimination of external data transmission and total data sovereignty.
Heterogeneous tasks require diverse computational capabilities	Deterministic capability-based routing across text LLM, VLM/OCR, dense vector RAG, and isolated data sandbox.	Optimal task-to-tool fit, minimized latency, and avoidance of resource overallocation.
Proprietary institutional knowledge must be queried without retraining	Dense local RAG pipeline with semantic chunking, local vector indexing, metadata filtering, and cross-encoder reranking.	Accurate, hallucination-resistant responses with page-level citations linked directly to internal evidence.
Autonomous agents can execute dangerous or infinite loops	Bounded agent state machine with strict step budgets (≤ 5 steps), tool allowlists, parameter sanitization, and automated timeouts.	Elimination of unbounded autonomous drift, runaway compute consumption, and unexpected tool calls.
Model-generated code can compromise host filesystem or network	Isolated execution environment with disabled networking, non-root user execution, memory limits, and temporary workspace teardown.	Hermetic containment of untrusted code; safe execution of complex analytical scripts.
Generative output can contain subtle hallucinations or arithmetic errors	Multi-stage verification layer with deterministic JSON schema checking, calculation validation, and human review for high-impact outputs.	High factual reliability and clear operational accountability before final delivery.
Organizations require compliance traceability and audit trails	Structured JSON runtime logging capturing model hyperparameters, tool invocations, token timings, and network activity.	Full forensic visibility, debuggability, and compliance with statutory security standards.

5.4 Innovation and Uniqueness of the Solution

The project does not claim novelty in developing novel foundation model architectures, as open-source LLMs, vector databases, and containers are existing technologies. Rather, the innovation lies in the **systems engineering and controlled integration architecture** tailored specifically for zero-trust organizational environments:

- **Capability-Based Routing over Monolithic Prompts:** Instead of sending every request to a massive general-purpose model, the router decomposes the task into discrete modalities (OCR, retrieval, calculation, reasoning) and routes each sub-task to the most efficient specialist engine.
- **Bounded Agency with Hard Safety Invariants:** Unlike unconstrained frameworks that give LLMs direct bash access, this system confines agent execution within an external finite state machine with strict step quotas, explicit tool allowlists, and deterministic safety invariants.
- **Multi-Stage Verification Layer:** By separating generation from verification, outputs undergo automated schema checks, retrieval coverage tests, and arithmetic validation, routing unverified claims to human reviewers.
- **Defense-in-Depth Local Security:** Security is implemented at every tier—from MIME-type upload validation and isolated sandbox execution to verifiable network silence monitoring.
- **Measurable Engineering Pragmatism:** The system establishes an honest boundary between demonstrated local capability and production requirements, prioritizing empirical benchmarking and explainability over exaggerated claims of artificial general intelligence.

5.5 Implementation Phases & Working Prototype Design

The project follows a disciplined nine-phase engineering lifecycle designed to ensure modular verification at each milestone:

- **Phase 1 — Threat Modeling & Boundary Definition:** Formulate threat models, define prohibited system actions, identify sensitive data categories, and establish human escalation thresholds.
- **Phase 2 — Local Inference Engine Deployment:** Deploy Ollama / llama.cpp runtime with quantized models (Qwen 2.5 7B, Llama 3.2) and benchmark CPU/VRAM throughput without internet connectivity.
- **Phase 3 — Task Analysis & Routing Engine:** Construct rule-based and embedding-based classifiers to route requests based on modality, estimated complexity, and hardware load.
- **Phase 4 — Multimodal Document Ingestion Pipeline:** Implement PyMuPDF and PaddleOCR extraction pipelines to parse complex multi-page PDFs, tabular data, and scanned equipment logs.
- **Phase 5 — Dense Local Knowledge Subsystem (RAG):** Implement recursive character chunking with metadata tagging, build a local ChromaDB/FAISS vector index, and integrate a cross-encoder reranker.
- **Phase 6 — Bounded Agent State Machine:** Implement an explicit state machine controlling agent planning, tool invocation, and state transitions, enforcing a maximum budget of 5 steps per task.
- **Phase 7 — Isolated Execution Sandbox:** Develop a network-disabled execution environment with strict CPU/memory quotas for executing Python scripts and data transformations.
- **Phase 8 — Verification Layer & Human Oversight:** Build automated output validators (schema matching, arithmetic cross-checking) and integrate an interactive human approval interface.
- **Phase 9 — Observability, Evaluation & Hardening:** Instrument structured JSON audit logging, monitor outbound network silence, and execute end-to-end benchmark evaluations on held-out industrial datasets.

Target Working Prototype Workflow

To rigorously validate the architecture, the working prototype demonstrates a high-impact, realistic industrial workflow: the automated ingestion, verification, and synthesis of confidential technical inspection reports:

1. **Document Ingestion & File Validation:** An operator submits a scanned multi-page PDF containing machinery sensor logs and operational notes. The gateway validates MIME integrity and initializes a unique audit session.
2. **Multimodal Extraction:** PaddleOCR parses image scans into text blocks while PyMuPDF extracts tabular metadata and structural headers, preserving page numbers and bounding coordinates.
3. **Policy-Governed Retrieval (RAG):** The agent queries internal standard operating procedures (SOPs) and safety compliance limits indexed within the local ChromaDB vector store, reranking results to obtain exact tolerance limits.
4. **Sandboxed Computational Analysis:** For tabular sensor readings, the agent writes a Python script to compute mean operating temperatures, pressure deviations, and anomaly percentages. The script executes within an air-gapped sandbox with no network access.
5. **Verification & Escalation:** The verification layer confirms that extracted values match the source scan and that calculated statistics satisfy schema invariants. If an anomaly exceeds safety thresholds, the system flags the issue for mandatory engineer sign-off.
6. **Deliverable Synthesis:** Upon approval, the system exports a tamper-evident audit report with page-level citations and chronological event logs in PDF/DOCX format.

6. Potential Impact on the Target Audience

The proposed on-premise workbench delivers substantial multidimensional benefits across diverse operational sectors:

Impact Dimension	Direct Operational & Societal Benefits	Engineering Contribution of the Solution
Social & Workforce Impact	Augments human engineers and administrative personnel by automating mundane document extraction, allowing specialists to focus on high-value cognitive decisions while retaining supervisory control.	Human-in-the-loop architecture guarantees that consequential technical actions require explicit human sign-off, preventing deskilling and loss of oversight.
Economic & Cost Viability	Eliminates recurring per-token subscription costs associated with public cloud APIs; provides predictable operational expense utilizing existing institutional compute hardware.	Capability-based routing prevents unnecessary activation of heavyweight models, maximizing throughput per watt and extending hardware lifecycle.
Security, Legal & Compliance	Guarantees strict compliance with the Indian Digital Personal Data Protection (DPDP) Act 2023, ISO 27001, and public-sector data classification mandates.	Zero network egress architecture mathematically eliminates remote data exfiltration, vendor tracking, and prompt telemetry exposure.
Environmental & Energy Efficiency	Drastically cuts unnecessary carbon footprints compared to querying 100B+ parameter hyperscale cloud models for simple internal tasks.	Employs quantized 7B/8B small language models (SLMs) and deterministic routing, minimizing unnecessary GPU power dissipation.
Institutional Knowledge Retention	Transforms unindexed, siloed archival records, technical manuals, and scanned legacy documents into active, searchable institutional intelligence.	Dense local RAG preserves organizational memory independently of external SaaS vendor availability or internet outages.

Primary Beneficiaries & Target Audience Segments

- **Industrial Manufacturing & Heavy Engineering:** Plant managers, safety inspectors, and quality assurance engineers who require rapid extraction and trend analysis from equipment logs, vibration scans, and calibration certificates without uploading proprietary blueprints to external clouds.
- **Public Sector & Government Administrative Departments:** Civil offices handling confidential citizen records, policy circulars, and departmental correspondence under strict data protection and air-gapped guidelines.
- **Legal & Corporate Compliance Firms:** Legal associates analyzing complex multi-hundred-page commercial contracts, patents, and nondisclosure agreements under strict attorney-client privilege.
- **Healthcare & Clinical Research Facilities:** Medical practitioners querying localized clinical records and treatment protocols without compromising patient confidentiality under health privacy regulations.

7. Tools, Technologies & Resource Requirements

7.1 Hardware Resource Specifications

Deployment Tier	Target Hardware Specifications	Operational Scope & Model Capabilities
Minimum Development Specification (Student Workstation)	CPU: Modern 6–8 Core x86_64 Processor RAM: 16 GB System Memory Storage: 512 GB NVMe SSD GPU: Integrated / Entry-level GPU (CPU-only fallback)	Enables local document extraction, dense RAG vector search, and execution of quantized 3B–7B SLMs (Q4_K_M quantization). Response generation is functional though token throughput is modest (~4–8 tokens/s on CPU).
Recommended MVP Specification (Dedicated Lab Station)	CPU: 8–16 Core High-Performance CPU RAM: 32–64 GB High-Speed DDR5 RAM Storage: 1 TB PCIe Gen4 NVMe SSD GPU: NVIDIA RTX 3060 / 4060 / 4070 (12–16 GB VRAM)	Enables simultaneous loading of 7B–14B instruction models, multimodal vision models (Qwen2-VL), dense embeddings, and real-time OCR parsing with interactive token generation (~25–45 tokens/s).
Production Scaled Reference (Enterprise On-Prem Node)	CPU: Dual AMD EPYC / Intel Xeon (32+ Cores) RAM: 128+ GB ECC RAM Storage: Multi-TB Enterprise NVMe RAID GPU: Dual NVIDIA RTX A6000 / A100 / L40S (48–80 GB VRAM)	High-concurrency enterprise deployment supporting continuous multi-user document pipelines, concurrent agent execution, and batch RAG indexing.

7.2 Software Stack and Architectural Libraries

Functional Layer	Technology / Framework	Specific Role & Implementation Details
Core Programming	Python 3.11+, HTML5 / Tailwind CSS, JavaScript	High-performance backend orchestration, asynchronous task handling, and responsive desktop interface.
Backend API Gateway	FastAPI, Uvicorn, Pydantic v2	Asynchronous REST service layer, strict schema validation, request authentication, and audit identifier propagation.
Local Model Runtime	Ollama, llama.cpp, vLLM	High-throughput local model serving, dynamic GPU layer offloading, context window management, and zero runtime API calls.
Foundation Models	Qwen 2.5 (7B/14B), Meta Llama 3.2 (3B/8B), Mistral NeMo	Quantized instruction-following models utilized for reasoning, synthesis, structured data extraction, and report drafting.
Multimodal & OCR	PaddleOCR, PyMuPDF (fitz), Qwen2-VL	Parsing of multi-page PDFs, high-accuracy tabular extraction, layout preservation, and visual chart comprehension.
Knowledge Embeddings	BAAI/bge-large-en-v1.5, all-MiniLM-L6-v2	Local dense vector embeddings for semantic document chunking and domain-specific knowledge representation.
Vector Storage	ChromaDB, FAISS (Facebook AI Similarity Search)	Fast, persistent, on-disk similarity search over organizational document embeddings with metadata filtering.
Reranking Engine	BAAI/bge-reranker-large	Cross-encoder reranking to re-score top candidate chunks, drastically enhancing RAG precision and context relevance.
Agent Orchestration	Custom State Machine Engine / LangGraph	Deterministic graph-based agent workflow enforcing maximum step limits, tool execution allowlists, and state checkpoints.
Isolated Sandbox	Rootless Subprocess Isolation / Podman Containers	Restricted runtime with dropped capabilities, disabled network loopback, and strict CPU/RAM memory quotas.
Document Generation	ReportLab, python-docx, openpyxl, pandas	Programmatic generation of structured executive reports, spreadsheet summaries, and publication-ready deliverables.
Metadata & Audit DB	SQLite 3 / PostgreSQL	Persistent storage of user session metadata, document provenance, verification flags, and chronological audit trails.

8. References (APA Format)

- [1] Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., ... & Kaplan, J. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*.
- [2] Dettmers, T., Svirschevski, R., Egiazarian, V., Kuzmin, A., Baziotis, C., & Zettlemoyer, L. (2023). SpQR: A sparse-quantized representation for near-lossless LLM weight compression. *arXiv preprint arXiv:2306.03078*.
- [3] Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... & Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.
- [4] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, 79-90.
- [5] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459-9474.
- [6] Liu, F., Lin, Z., Shen, C., & Ding, M. (2023). On the safety and sandbox isolation of autonomous agent execution in local environments. *IEEE Transactions on Dependable and Secure Computing*, 21(3), 1420-1435.
- [7] Microsoft Corporation. (2024). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework*. Microsoft Research Technical Report.
- [8] Ministry of Law and Justice, Government of India. (2023). *The Digital Personal Data Protection Act, 2023 (No. 22 of 2023)*. The Gazette of India Extraordinary.
- [9] Schick, T., Dwivedi-Yu, J., Dessi, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., ... & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 68539-68551.
- [10] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., ... & Scialom, T. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- [11] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., ... & Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations (ICLR)*.
- [12] Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., & Zhou, M. (2021). LayoutLMv2: Multi-modal pre-training for visually-rich document understanding. *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2579-2591.
- [13] Yang, A., Xiao, B., Wang, B., Zhang, B., Bian, C., Yin, C., ... & Zhou, J. (2024). Baichuan 2: Open large-scale language models. *arXiv preprint arXiv:2309.10305*.
- [14] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*.
- [15] Zhang, S., Dong, L., Li, X., Zhang, S., Sun, X., Wang, S., ... & Wei, F. (2023). Instruction tuning for large language models: A survey. *arXiv preprint arXiv:2308.10792*.

DEPARTMENTAL REVIEW & SYNOPSIS APPROVAL

Bhumik Sharma
(2401920100127)

Harsh Kumar Singh
(2401920100152)

Harsh Kumar
(2401920100151)

Harshit Kumar
(2401920100155)

Ms. Abha Kaushik
Project Supervisor / Guide

Project Coordinator / HOD
Department of CSE, GLBITM